

N+1 problem

What is the N+1 Problem?

- **Definition:**
The N+1 problem occurs when your ORM (Hibernate/JPA) runs 1 query for the parent entity and then N queries (one per row) for the child/related entities.
- **Why it happens:**
Because of lazy loading (FetchType.LAZY) in JPA. Lazy loading means related entities are not fetched until you actually access them.

Example of N+1

Imagine you want to fetch all posts and their authors.

- **Query 1:** `SELECT * FROM Post` → gets 100 posts.
- Then, for each post, Hibernate fires:
 - `SELECT * FROM Author WHERE id=?` (100 times).

Why is N+1 Bad?

- **Performance issue:**
 - Works fine for small data.
 - But with thousands of rows → database becomes a bottleneck.
- **Hard to detect:** unless you log SQL queries.
- **Not obvious in dev** (works with 5 rows but fails in prod with 50k).

How to Fix N+1 in Java

a) JOIN FETCH (Eager Loading with JPQL/HQL)

Tell Hibernate to join related tables in one query.

- Fixes N+1 by fetching parent + child in a single SQL join.

b) Entity Graphs (JPA 2.1)

Declarative way to specify fetch plans.

- Lets you define which relationships should be loaded eagerly for a query.
- More flexible than hardcoding JOIN FETCH in JPQL.

c) Batch Fetching (Hibernate feature)

Instead of firing 1 query per entity, Hibernate groups them in batches.

- Example: If batch-size=20 → Hibernate fetches 20 related entities in one query.
- Reduces N queries → N/20 queries.

d) Second Level Cache

- Cache frequently used entities/relationships (EhCache, Hazelcast, Redis).
- Reduces repetitive queries but does not eliminate N+1 directly.

e) Best Practices

- Use JOIN FETCH for **one-to-one** and **many-to-one** relations.
- Use LEFT JOIN FETCH + DISTINCT for **one-to-many**.
- Enable **SQL logging** in dev (hibernate.show_sql, p6spy) to catch N+1.
- Use **pagination** for large datasets.

1. Basic — Show N+1 Issue (Lazy Loading)

Goal:

- Demonstrate how Hibernate generates **1 query for parent + N queries for children**.

Steps:

1. Create two entities: Author and Post (@OneToMany + @ManyToOne).
2. Use FetchType.LAZY (default).
3. Write code:
 - Fetch all posts.
 - Access each post's author.
4. Observe SQL logs → 1 query for posts + N queries for authors.

2. Intermediate — Fix with JOIN FETCH

Goal:

- Solve N+1 using **JPQL with JOIN FETCH**.

Steps:

1. Modify query:
2. `SELECT p FROM Post p JOIN FETCH p.author`
3. Fetch posts with their authors in **one query**.
4. Run the same loop → Hibernate won't fire extra queries.

3. Advanced — Batch Fetching & Entity Graphs

a) Batch Fetching

- Configure in `application.properties` or `persistence.xml`:
- `spring.jpa.properties.hibernate.default_batch_fetch_size=20`
- Hibernate now loads related entities in **chunks** instead of one-by-one.

b) Entity Graphs (JPA 2.1)

- Create an **EntityGraph** for flexible fetching:

```
EntityGraph<Post> graph = em.createEntityGraph(Post.class);
graph.addAttributeNodes("author");

List<Post> posts = em.createQuery("SELECT p FROM Post p", Post.class)
    .setHint("javax.persistence.fetchgraph", graph)
    .getResultList();
```

- EntityGraphs give fine-grained control without hardcoding JPQL.

4. Pro — Monitoring & Logging

Goal:

- Prove performance improvement by **tracking queries**.

Steps:

1. Enable SQL logging:

```
spring.jpa.show-sql=true
logging.level.org.hibernate.SQL=DEBUG
logging.level.org.hibernate.type.descriptor.sql.BasicBinder=TRACE
```

2. Use **p6spy** or **Datasource-Proxy** for query count.

3. Run all versions:

- N+1 (baseline) → ~101 queries.
- JOIN FETCH → 1 query.
- Batch Fetch → ~6 queries (if 100 entities, batch size 20).
- Entity Graph → 1 query.

pom.xml (dependencies)

```
<dependencies>

  <!-- Spring Boot JPA -->

  <dependency>

    <groupId>org.springframework.boot</groupId>

    <artifactId>spring-boot-starter-data-jpa</artifactId>

  </dependency>

  <!-- H2 In-Memory DB -->

  <dependency>

    <groupId>com.h2database</groupId>

    <artifactId>h2</artifactId>

    <scope>runtime</scope>

  </dependency>
```

```
<!-- p6spy for SQL logging (optional, for Pro stage) -->
<dependency>
  <groupId>p6spy</groupId>
  <artifactId>p6spy</artifactId>
  <version>3.9.1</version>
</dependency>

<!-- Spring Boot Starter Web (to run simple REST endpoints) -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
</dependencies>
```

Entities

Author.java

```
import jakarta.persistence.*;
import java.util.*;
```

```
@Entity
public class Author {
  @Id
  @GeneratedValue
  private Long id;

  private String name;
```

```
@OneToMany(mappedBy = "author", fetch = FetchType.LAZY, cascade = CascadeType.ALL)
```

```
private List<Post> posts = new ArrayList<>();
```

```
// getters & setters
```

```
public Long getId() { return id; }
```

```
public void setId(Long id) { this.id = id; }
```

```
public String getName() { return name; }
```

```
public void setName(String name) { this.name = name; }
```

```
public List<Post> getPosts() { return posts; }
```

```
public void setPosts(List<Post> posts) { this.posts = posts; }
```

```
}
```

Post.java

```
import jakarta.persistence.*;
```

```
@Entity
```

```
public class Post {
```

```
    @Id
```

```
    @GeneratedValue
```

```
    private Long id;
```

```
    private String title;
```

```
    @ManyToOne(fetch = FetchType.LAZY)
```

```
    private Author author;
```

```
// getters & setters

public Long getId() { return id; }

public void setId(Long id) { this.id = id; }


public String getTitle() { return title; }

public void setTitle(String title) { this.title = title; }


public Author getAuthor() { return author; }

public void setAuthor(Author author) { this.author = author; }

}
```

Repository Layer

AuthorRepository.java

```
import org.springframework.data.jpa.repository.JpaRepository;

public interface AuthorRepository extends JpaRepository<Author, Long> {

}
```

PostRepository.java

```
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import java.util.List;

public interface PostRepository extends JpaRepository<Post, Long> {
```

```

// Intermediate: JOIN FETCH to fix N+1
@Query("SELECT p FROM Post p JOIN FETCH p.author")
List<Post> findAllWithAuthors();

}

```

Service Layer

NPlusOneService.java

```

import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.util.List;

@Service
public class NPlusOneService {

    private final PostRepository postRepository;

    public NPlusOneService(PostRepository postRepository) {
        this.postRepository = postRepository;
    }

    // Basic: N+1 Problem
    @Transactional(readOnly = true)
    public void showNPlusOneProblem() {
        System.out.println("\n--- BASIC: N+1 Problem ---");
        List<Post> posts = postRepository.findAll(); // 1 query for posts
        for (Post post : posts) {
            // Triggers extra queries for authors
            System.out.println(post.getTitle() + " by " + post.getAuthor().getName());
        }
    }
}

```



```

    }
}

// Intermediate: Fix with JOIN FETCH

@Transactional(readOnly = true)
public void fixWithJoinFetch() {
    System.out.println("\n--- INTERMEDIATE: JOIN FETCH Fix ---");
    List<Post> posts = postRepository.findAllWithAuthors(); // 1 query with join
    for (Post post : posts) {
        System.out.println(post.getTitle() + " by " + post.getAuthor().getName());
    }
}
}
}

```

AdvancedService.java

```

import jakarta.persistence.EntityGraph;
import jakarta.persistence.EntityManager;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class AdvancedService {

    private final EntityManager em;

    public AdvancedService(EntityManager em) {
        this.em = em;
    }
}

```

```
// Advanced: Entity Graph

public void fixWithEntityGraph() {

    System.out.println("\n--- ADVANCED: Entity Graph Fix ---");

    EntityGraph<Post> graph = em.createEntityGraph(Post.class);
    graph.addAttributeNodes("author");

    List<Post> posts = em.createQuery("SELECT p FROM Post p", Post.class)
        .setHint("javax.persistence.fetchgraph", graph)
        .getResultList();

    for (Post post : posts) {
        System.out.println(post.getTitle() + " by " + post.getAuthor().getName());
    }
}
}
```

Data Initialization

DataLoader.java

```
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;

@Component

public class DataLoader implements CommandLineRunner {

    private final AuthorRepository authorRepository;

    public DataLoader(AuthorRepository authorRepository) {
        this.authorRepository = authorRepository;
    }

    @Override
    public void run(String... args) {
        for (int i = 1; i <= 5; i++) {
            Author author = new Author();
            author.setName("Author " + i);
            for (int j = 1; j <= 5; j++) {
                Post post = new Post();
                post.setTitle("Post " + j + " of Author " + i);
                post.setAuthor(author);
                author.getPosts().add(post);
            }
            authorRepository.save(author);
        }
    }
}
```

Application Runner

NPlusOneDemoApplication.java

```
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication

public class NPlusOneDemoApplication implements CommandLineRunner {

    private final NPlusOneService service;

    public NPlusOneDemoApplication(NPlusOneService service) {
        this.service = service;
    }

    public static void main(String[] args) {
        SpringApplication.run(NPlusOneDemoApplication.class, args);
    }

    @Override
    public void run(String... args) {
        service.showNPlusOneProblem(); // Basic
        service.fixWithJoinFetch();   // Intermediate
        // TODO: Advanced (Batch Fetching + Entity Graph)
    }
}
```

Advanced

Batch Fetching

```
spring.jpa.show-sql=true
```

```
spring.jpa.properties.hibernate.format_sql=true
```

```
spring.jpa.properties.hibernate.default_batch_fetch_size=20
```

Entity Graph Example

```
import jakarta.persistence.EntityGraph;
```

```
import jakarta.persistence.EntityManager;
```

```
import org.springframework.stereotype.Service;
```

```
@Service
```

```
public class AdvancedService {
```

```
    private final EntityManager em;
```

```
    public AdvancedService(EntityManager em) {
```

```
        this.em = em;
```

```
    }
```

```
    public void fixWithEntityGraph() {
```

```
        System.out.println("\n--- Entity Graph Fix ---");
```

```
        EntityGraph<Post> graph = em.createEntityGraph(Post.class);
```

```
        graph.addAttributeNodes("author");
```

```

List<Post> posts = em.createQuery("SELECT p FROM Post p", Post.class)
    .setHint("javax.persistence.fetchgraph", graph)
    .getResultList();

for (Post post : posts) {
    System.out.println(post.getTitle() + " by " + post.getAuthor().getName());
}
}
}

```

Pro — Monitoring

- With `spring.jpa.show-sql=true` → you'll see query counts.
- With **p6spy** → you get exact executed SQL queries.

Run the app → compare queries fired in each method:

- **N+1 Problem** → ~26 queries (1 for posts + 25 for authors).
- **JOIN FETCH** → 1 query.
- **Batch Fetch** → fewer queries depending on batch size.
- **Entity Graph** → 1 query.